

CLASSES

Almost everything in Python is an object, with its properties and methods. A class is like an object constructor, or a 'blueprint' for creating objects.

To create a car, we can't start building right away. Firstly, we need to create a blueprint of the car.

In Python, the class is like the blueprint of the object, car.

The blueprint allows to create as many cars as it wants. In the same way, we can create as many objects as we want from a class.

```
#SYNTAX:
#class ClassName:
#    #Statements..
class Student:
    pass

print(Student)

<class '__main__.Student'>
```

Creating Objects in the class

Also known as object instantiation

Objects consists of identity, state and behaviour.

Identity: Name of the object

State: unique state of object is by creating attributes in the **init** method of the class.

Behaviour: What functions it could perform on attributes.

```
##SYNTAX: ** Objectname= Classname(*args)
Rutuja= Student()

print(Rutuja)

<__main__.Student object at 0x7f80f1140940>

i= int()
print(type(i))

<class 'int'>
```

Dot operator(.) helps to access variables and functions inside the class.

```
#SYNTAX: object_name.attribute_name
#SYNTAX: object_name.method_name(*args)

class Student:
    college='SIT'

Rutuja=Student()
print(Rutuja.college)

SIT

class Student:
    college='SIT'

Rutuja=Student()
Pooja=Student()
print(Rutuja.college)
print(Pooja.college)

#Its a class attribute:
print(Student.college)

SIT
SIT
SIT
```

Changing the value of a class attribute using the class name causes the value change in all objects of the class.

Double-click (or enter) to edit

```
Student.college = 'SIT Nagpur'  
print(Student.college)  
print(Rutuja.college)  
print(Pooja.college)
```

```
SIT Nagpur  
SIT Nagpur  
SIT Nagpur
```

In Python, a **constructor** is a special method used for initializing objects. It's called automatically when you create a new instance of a class. In Python, the constructor method is named **init()**.

The `__init__()` Function

The examples above are classes and objects in their simplest form, and are not really useful in real life applications.

To understand the meaning of classes we have to understand the built-in `__init__()` function.

All classes have a function called `__init__()`, which is always executed when the class is being initiated.

Use the `__init__()` function to assign values to object properties, or other operations that are necessary to do when the object is being created:

Example

Create a class named Person, use the `__init__()` function to assign values for name and age:

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
p1 = Person("John", 36)  
  
print(p1.name)  
print(p1.age)
```

Activate
Go to Set

The self Parameter

The `self` parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.

It does not have to be named `self`, you can call it whatever you like, but it has to be the first parameter of any function in the class:

Example

Use the words *mysillyobject* and *abc* instead of *self*:

```
class Person:
    def __init__(mysillyobject, name, age):
        mysillyobject.name = name
        mysillyobject.age = age

    def myfunc(abc):
        print("Hello my name is " + abc.name)

p1 = Person("John", 36)
p1.myfunc()
```

Activa

Objects can also contain methods. Methods in objects are functions that belong to the object. The function above prints a greeting, and execute it on the p1 object

```
class Complex:
    def __init__(self, realpart, imagpart):
        self.r=realpart
        self.i=imagpart
x=Complex(3.0,-4.5)
print('Complex no is:', x.r, '+ i', x.i)
```

Complex no is: 3.0 + i -4.5

-Variables that belong to an object or class are referred to as fields. Objects can also have functionality by using functions that belong to a class. Such functions are called methods of the class. Collectively, the fields and methods can be referred to as the attributes of that class.

-Fields are of two types: they can belong to each instance/object of the class or they can belong to the class itself. They are called **instance variables** and **class variables** respectively.

```
class Student:
    college='SIT'          #Class Variable
    branch='CSE'
    def __init__(self,name,m1,m2,m3):
        self.div='B'      # Instance Variable
        self.name=name    # Instance Variable
        self.m1=m1
        self.m2=m2
        self.m3=m3
    def percentage(self):
        total=self.m1+self.m2+self.m3
        perc=(total/300)*100
        print("The percentage is ",perc)
    def Display(self):
        print("The college is ", self.college)
        print("The Branch is ", self.branch)
        print("The division is ",self.div)
        print("The name is ",self.name)
# Object of Class Student
s1=Student('SaraI',50,60,70)
s1.Display()          # This is equivalent to student.Display(s1)
s1.percentage()
```

The college is SIT
The Branch is CSE
The division is B
The name is SaraI
The percentage is 60.0

del is a destructor method which is called as soon as all references of the object are deleted

```
class Student:
    college='SIT'          #Class Variable
    branch='CSE'
    def __init__(self,name,m1,m2,m3):
        self.div='B'      # Instance Variable
        self.name=name    # Instance Variable
        self.m1=m1
        self.m2=m2
        self.m3=m3

    def percentage(self):
        total=self.m1+self.m2+self.m3
        perc=(total/300)*100
        print("The percentage is ",perc)
    def Display(self):
        print("The college is ", self.college)
        print("The Branch is ", self.branch)
        print("The division is ",self.div)
        print("The name is ",self.name)
    def __del__(self):
        print("The object is deleted")

# Object of Class Student
s1=Student('Saral',50,60,70)
s1.Display()      # This is equivalent to student.Display(s1)
s1.percentage()
del s1
```

```
The college is  SIT
The Branch is  CSE
The division is  B
The name is  Saral
The percentage is  60.0
The object is deleted
```

```
s1.Display()
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-32-238b377402cd> in <cell line: 1>()
----> 1 s1.Display()

NameError: name 's1' is not defined
```

INHERITANCE:

Inheritance is a feature or a process in which, new classes are created from the existing classes.

```
class Student:
    college='SIT'
    c_fees=20000

    def Display(self):
        print("The college name is : ",self.college)
        print("The college fees is : ",self.c_fees)

Mridula=Student()
Mridula.Display()

The college name is :  SIT
The college fees is :  20000

class Hostler:
    h_fees=10000
    def __init__(any_name,hostel_room_no):
        any_name.branch='CSE'
        any_name.hostel_room_no=hostel_room_no
    def Display(var):
        print("The hostel fees is ", var.h_fees)
        print("The Branch is ", var.branch)
        print("The hostel room no. is ",var.hostel_room_no)

s1=Hostler('G11')
s1.Display()
```

```
The hostel fees is 10000
The Branch is CSE
The hostel room no. is G11
```

```
class Hostler(Student): # single level inheritance (one base class (student) and one child class(hostler))
    h_fees=10000
    def __init__(any_name,hostel_room_no):
        any_name.branch='CSE'
        any_name.hostel_room_no=hostel_room_no
    def Display(var):
        print("The hostel fees is ", var.h_fees)
        print("The Branch is ", var.branch)
        print("The hostel room no. is ",var.hostel_room_no)
s1=Hostler('G11')
print(s1.college)
print("The college fees is : ", s1.c_fees)
s1.Display()
```

```
SIT
The college fees is : 20000
The hostel fees is 10000
The Branch is CSE
The hostel room no. is G11
```

Method overriding with multiple and multilevel inheritance.

Method overriding refers to the ability to define a method in a subclass (child class) that has the same name, parameters, and return type as a method in its superclass (parent class). When a method is overridden in a subclass, the implementation in the subclass is used instead of the implementation in the superclass when the method is called on an instance of the subclass.

Multilevel Inheritance: When we have a child and grandchild relationship. Example: Let's consider an example where we want to override only one method of one of its parent classes

```
class Parent():
    # Parent's show method
    def display(self):
        print("Inside Parent")

# Inherited or Sub class
class Child(Parent):
    # Child's show method
    def show(self):
        print("Inside Child")

# Inherited or Sub class (Note Child in bracket)
class GrandChild(Child):
    # Child's show method
    def show(self):
        print("Inside GrandChild")

g = GrandChild()
g.show()
g.display()

    Inside GrandChild
    Inside Parent
```

Multiple Inheritance: When a class is derived from more than one base class it is called multiple Inheritance. Example: Let's consider an example where we want to override a method of one parent class only

```
class Parent1():
```